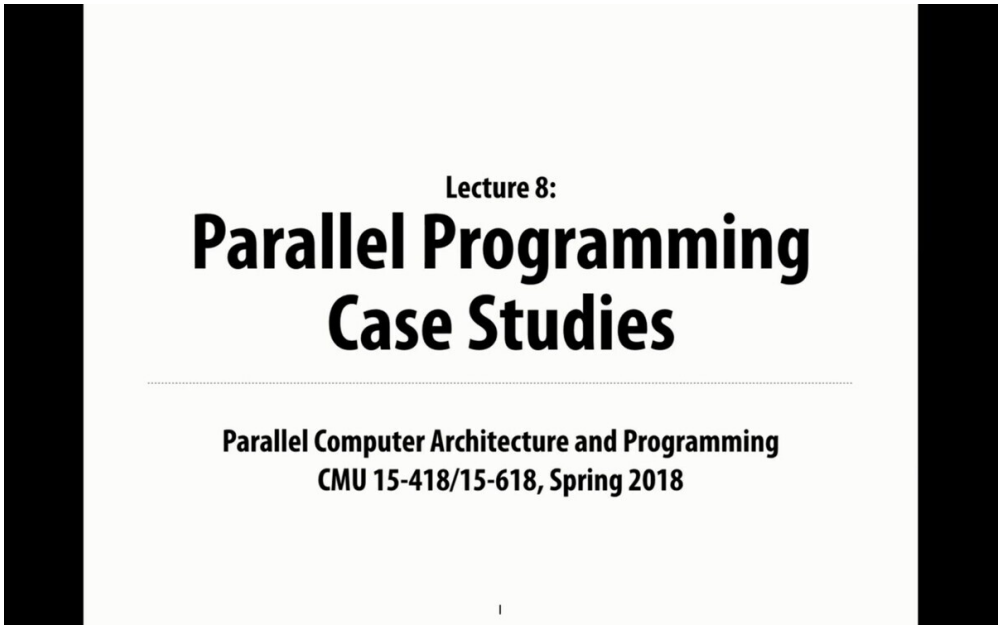
课程笔记

并行程序案例研究：海洋模拟与星系演化

CMU 15-418/618 并行计算机架构与编程 · Lecture 9（Spring 2018）

lecture-to-notes

2026 年 10 月 29 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 20 分 48 秒
视频链接：`https://www.bilibili.com/video/BV1ej41167sy/`

目录

1 从原理转向案例 **2**
1.1 两个核心关切 . . . 2
1.2 案例的来源 . . . 2
1.3 本章小结 . . . 3

2 硬件假设：NUMA **3**
2.1 统一的思考模型 . . . 3
2.2 具体机器 . . . 4
2.3 本章小结 . . . 4

3 案例一：海洋模拟 **5**
3.1 问题设定 . . . 5
3.2 为什么这个问题难以扩展 . . . 5
3.3 核心计算与两类工作 . . . 6
3.4 本章小结 . . . 6

4 网格划分：面积与周长之比 **6**
4.1 为什么越方越好 . . . 6
4.2 屏障与掉队者 . . . 8
4.3 工作集的时间尺度 . . . 9
4.4 静态分配的适用条件与实测 . . . 10
4.5 本章小结 . . . 11

5 案例二：星系演化与 N 体问题 **12**
5.1 为什么比网格问题难 . . . 12
5.2 $O(n^2)$ 与它的出路 . . . 12
5.3 本章小结 . . . 13

6 Barnes-Hut 树 **14**
6.1 树的构造 . . . 14
6.2 质心的逐层合成 . . . 14
6.3 建树与遍历的代价 . . . 15
6.4 本章小结 . . . 16

7 负载不均与动态划分 **16**
7.1 问题的形态 . . . 16
7.2 不能按数量分配 . . . 16
7.3 用代价区域分配 . . . 17
7.4 工作集视角下的树遍历 . . . 17
7.5 本章小结 . . . 18

8 总结与延伸 **18**

1 从原理转向案例

这一讲开始用真实程序演示前几讲的原理如何落地。讲者把课程位置交代得很清楚：到目前为止讨论的是一般原则——如何思考并行计算、它的执行部分与通信部分；接下来用一些例子说明这些问题在不同领域是怎么被解决的，然后才进入如何测量与评估系统性能。

1.1 两个核心关切

讲者点明这一讲要看的重点：这些程序用了什么技术来提取并行、处理我们已经谈过的问题。最主要的两个是保证处理器之间有合理的负载均衡，以及避免通信带来的过高开销。



图 1: 课程路线图：从原理转向案例研究[1]

1.2 案例的来源

这些案例来自课程网页上列出的一本教材。讲者说那本书已有二十年历史，但其中的原则与概念在今天是普遍有效的。

他的判断值得记录：除了 GPU 的出现之外，并行计算领域在过去二十年里概念上没有太大变化，变的只是可行的规模。

[1] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

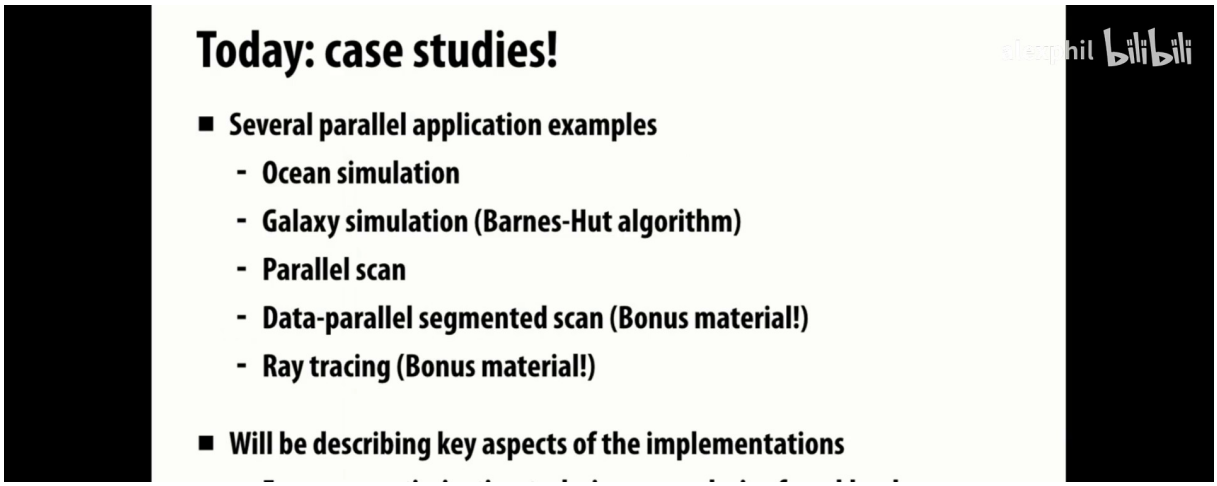


图 2: 今日主题：几个并行应用案例[2]

今天要看的有四个：海洋模拟（典型的网格求解问题）、星系建模（不规整的空间划分问题）、从扫描推广到分段扫描、以及光线追踪与图形学。

1.3 本章小结

这一讲用真实程序演示两个核心关切：负载均衡与通信开销。案例取自二十年前的教材，但概念至今有效——除 GPU 外，并行计算的变化主要在规模而非原理。

下一章先明确硬件假设，因为后面的划分策略都建立在这个假设上。

2 硬件假设：NUMA

2.1 统一的思考模型

讲者说为了讨论方便，请大家统一按一个模型思考：大规模共享内存多处理器。也就是说，通信上可以寻址整个内存空间。



图 3: 假设：共享地址空间的多处理器[3]

[2] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

但这个共享内存是 NUMA 的——非一致内存访问，意思是某些部分的内存比其他部分更容易、更快地访问到。

原因有两层：一是缓存效应，二是物理构造方式。一旦超出单芯片多核处理器，通常的做法是各个处理器分别管理一段 DRAM，可以跨过去请求访问，但要经过另一颗芯片的 DRAM 控制器。

NUMA 让”内存”这个词失去单数含义

在 NUMA 机器上，一次访问的代价取决于地址落在哪个节点。这意味着数据布局不只是缓存友好性问题，而是直接决定了通信是否跨越慢速链路。因此后面所有划分策略的共同目标都是让大部分访问落在本地节点。

2.2 具体机器

讲者给出下一份作业会用到的集群：每个处理器是 12 核 Xeon，但物理上其实是两颗芯片。因此一半内存比另一半更快。

NUMA 是共享内存与消息传递之间的过渡形态

共享内存中一次访问的代价由地址决定；消息传递中一次通信的代价由显式发送决定；NUMA 介于两者之间——编程模型仍是共享地址空间，但代价已随位置变化。因此 NUMA 上的优化技术（本地性优先、按节点划分）在消息传递中大多仍适用。

他也说明这些想法可以推广到消息传递、分布式地址空间，以及在较小程度上推广到 GPU。

2.3 本章小结

硬件假设是大规模共享内存多处理器，且为 NUMA：部分内存访问更快。原因是缓存效应与物理构造——跨芯片访问要经对方的 DRAM 控制器。集群中每个处理器是 12 核 Xeon 但物理上两颗芯片，因此一半内存更快。这些思想可推广到消息传递与分布式地址空间。

[3] 视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

3 案例一：海洋模拟

3.1 问题设定



图 4: 模拟洋流：把海洋离散成二维网格，只取一个水平切片[4]

目标是模拟洋流。讲者说像许多这类问题一样，我们不必真的懂背后的科学，只需对计算类型有感觉。

做法与之前看过的类似：把空间离散成网格，只考虑海洋的一个水平切片——比如海面或某个特定深度。因此今天只把它当作二维问题。

要建模的是状态参数：温度、含氧量以及其他对海洋重要的量，看它们如何在时间与空间上演化。典型做法是设定一个小的时步 Δt，每隔 Δt 更新一次系统状态。

3.2 为什么这个问题难以扩展

讲者给出一个关键约束：对任何物理模拟来说，想要更细的分辨率、更细的空间分布，就必须同时缩短 Δt。

原因是 Δt 必须小到让效应在一个时间步内不传播超过最近邻。

他举了一个同行的例子：做地面运动、地震类模拟的人说，每当他们想把空间分辨率提高一倍，就需要 16 倍的算力。这是因为要在三个空间维度上都变细，同时时间也要变细。

$$\text{算力} \propto \left(\frac{1}{\Delta x}\right)^3 \cdot \frac{1}{\Delta t}$$

- Δx：空间步长，三个维度同时缩小；
- Δt：时间步长，受 Δx 约束而必须同比缩小；
- 若 Δx 减半，则空间项变为 $2^3 = 8$ 倍，时间项变为 2 倍，合计 16 倍。

分辨率提升的代价是四次的而非线性的

这是所有物理模拟共同面对的现实：提高精度不是多花一点算力，而是按四次方增长。这解释了为什么这类问题”非常难以扩展”，也解释了为什么超算排名长期被这类应用主导。

[4]视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

3.3 核心计算与两类工作

讲者说只关注图中标红的部分，也就是网格求解部分。它是我们上次讨论内容的稍复杂版本：在每个空间点上，计算该点与四个邻居的平均值。

当然实际计算比求平均更有意思，但这个计算是”对系统状态及其演化建模”这类工作的良好代表。典型做法是迭代直到满足某个收敛条件。

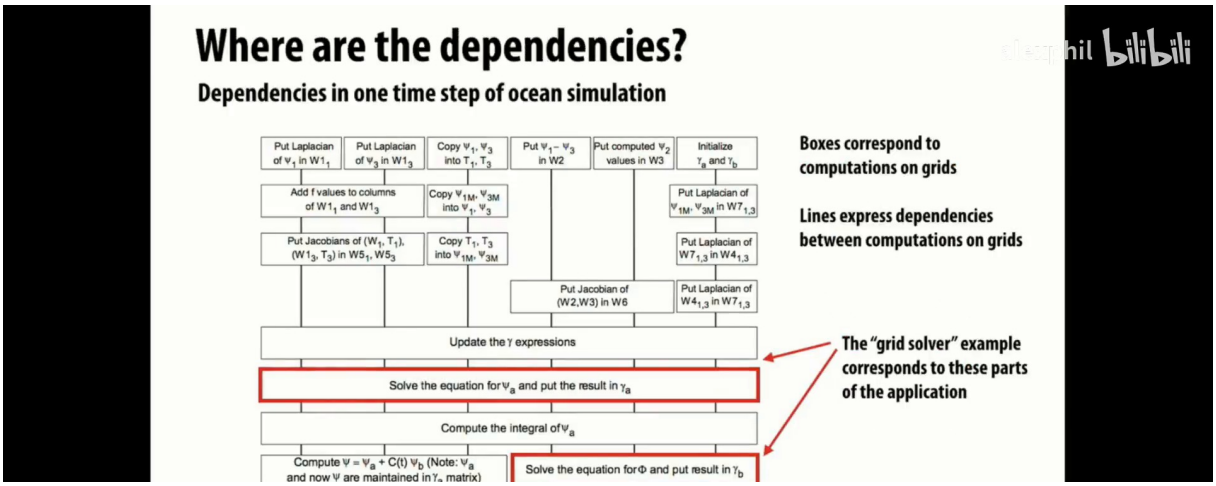


图 5: 依赖在哪里：一个时间步内海洋模拟的依赖关系[5]

与之前一样，这里有两类计算：一是更新每个元素；二是确定与上一步相比的变化量，并把这个变化信息聚合到某个中心位置，由它判断模拟是否收敛。

3.4 本章小结

海洋模拟把空间离散成二维网格（实际是三维的一个切片），状态参数含温度、含氧量等，按小时步更新。关键约束是提高空间分辨率必须同时缩短时间步，因此分辨率翻倍需要 16 倍算力。核心计算是求该点与四邻居的平均值，代表状态更新类计算；每个时间步包含两类工作：逐元素更新，以及把变化量归约到中心判断收敛。

下一章讨论如何把这个二维网格划分到多个处理器上。

4 网格划分：面积与周长之比

4.1 为什么越方越好

讲者回顾上一讲的结论：最好的做法是把空间划分成瓦片，而且瓦片越接近方形越好。

通信量由边界数据量决定：

$$V_{\text{comm}} = P \cdot w, \qquad V_{\text{comp}} = A$$

- P：瓦片周长，即需要交换的边界长度；
- w：每个边界元素需要传输的字节数（含光晕层）；
- A：瓦片面积，即该处理器负责的元素数。

[5] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。

因此每字节计算量正比于 $A/(P \cdot w)$，最大化它就等于最小化周长面积比。

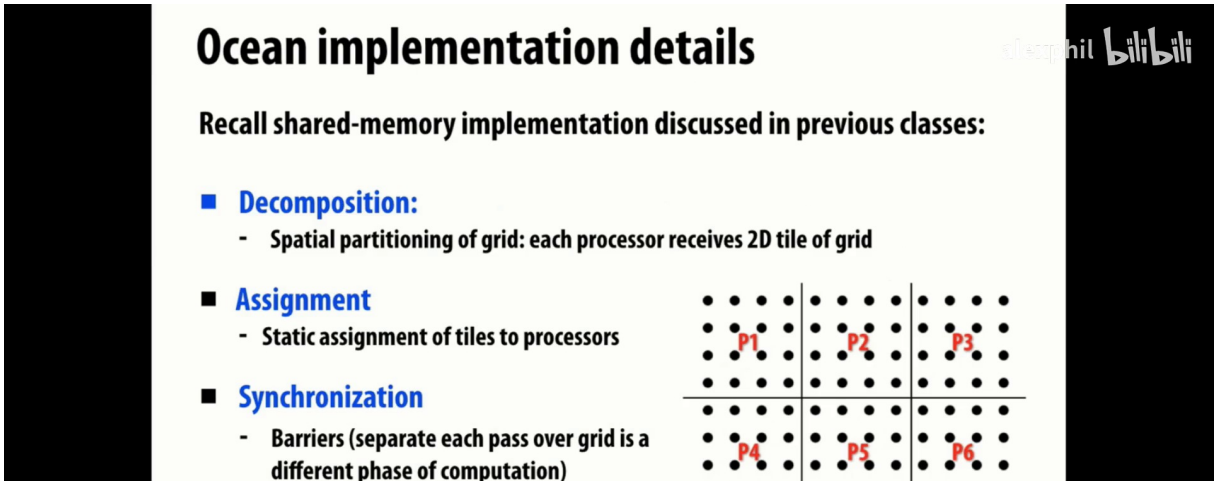


图 6: 海洋模拟的实现细节：共享内存实现要点[6]

理由很直接：计算量正比于面积，通信量正比于周长。因此要提高算术强度，就要让面积相对于周长尽量大，而正方形（或其他非常紧凑的形状）能最小化周长与面积之比。

$$\text{算术强度} \propto \frac{A}{P}, \qquad \left.\frac{A}{P}\right|_{\text{方形}} = \frac{s^2}{4s} = \frac{s}{4}$$

- A：瓦片面积，决定该处理器要做的计算量；
- P：瓦片周长，决定需要交换的边界数据量；
- s：方形瓦片的边长，因此瓦片越大，算术强度越高。

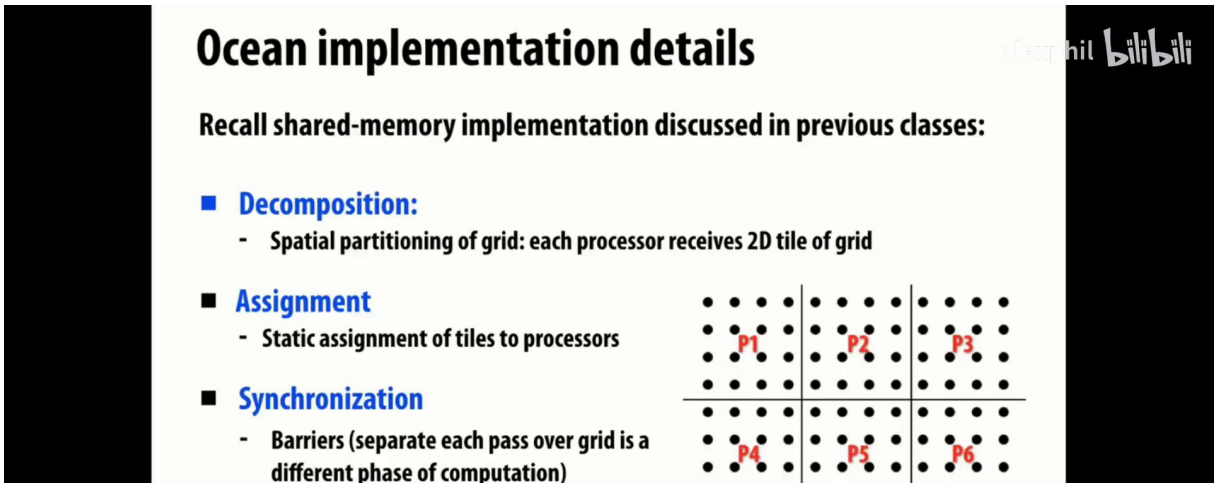


图 7: 实现细节续：分块与屏障同步[7]

[6] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。
[7] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

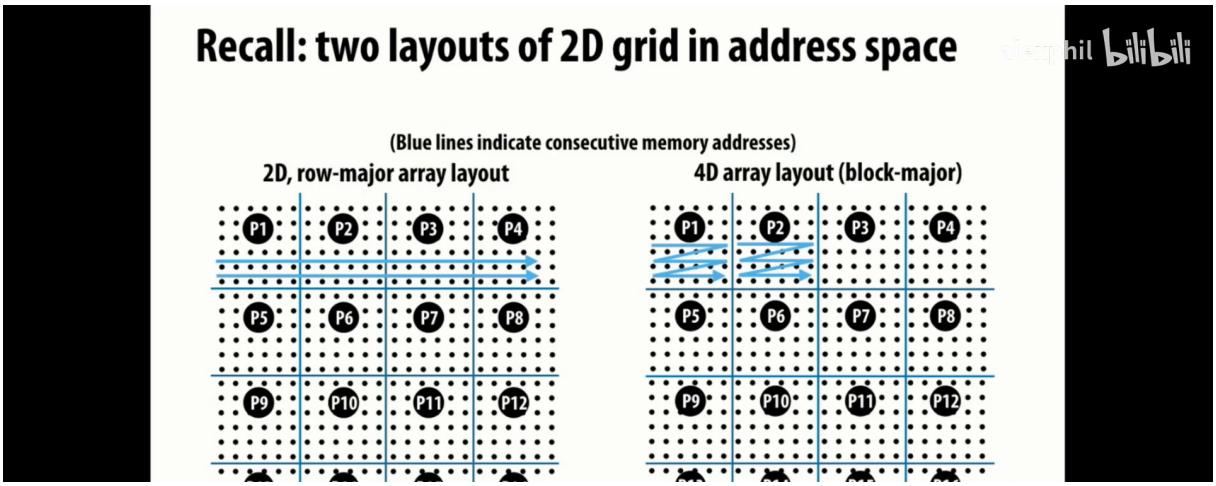


图 8: 回顾：两种静态划分方案——块划分与条带划分[8]

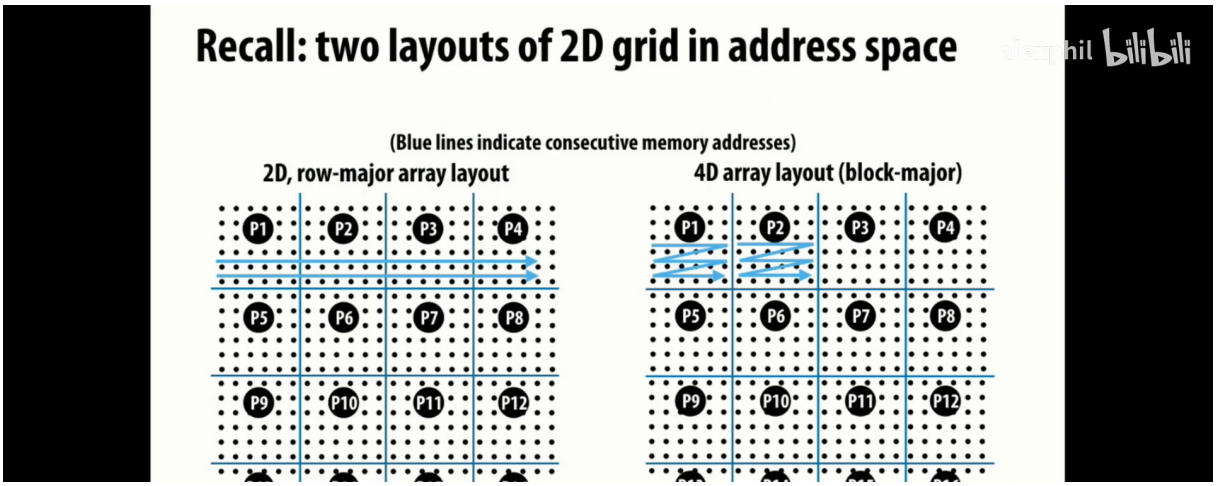


图 9: 回顾：二维网格在地址空间中的两种布局，蓝线表示连续地址[9]

4.2 屏障与掉队者

讲者接着讨论屏障同步的成本。它有两个代价。

第一是让所有人就”该停下来了”达成一致所需的时间。第二，若某个元素跑得比其他人慢一点——这叫掉队者（straggler）——它会迫使所有人按它的节奏走。

掉队者的代价随同步频率放大

若每步都同步，则总耗时约等于步数乘以最慢者的单步耗时。因此减少同步次数（增大每步工作量）可以把掉队者的影响摊薄，代价是并行粒度变粗、负载均衡更难。这是同步型应用的核心权衡之一。

讲者说这在这类计算中是个普遍的大问题。

[8] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

[9] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

屏障把最慢者的延迟放大为全局延迟

在每步都要同步的结构里，单步耗时不取决于平均值，而取决于最大值。因此负载不均的影响不是”稍微慢一点”，而是”整台机器按最慢分区的速度运行”。这解释了为什么网格划分要追求均衡，以及为什么后面 N 体的动态划分是必要的。

4.3 工作集的时间尺度

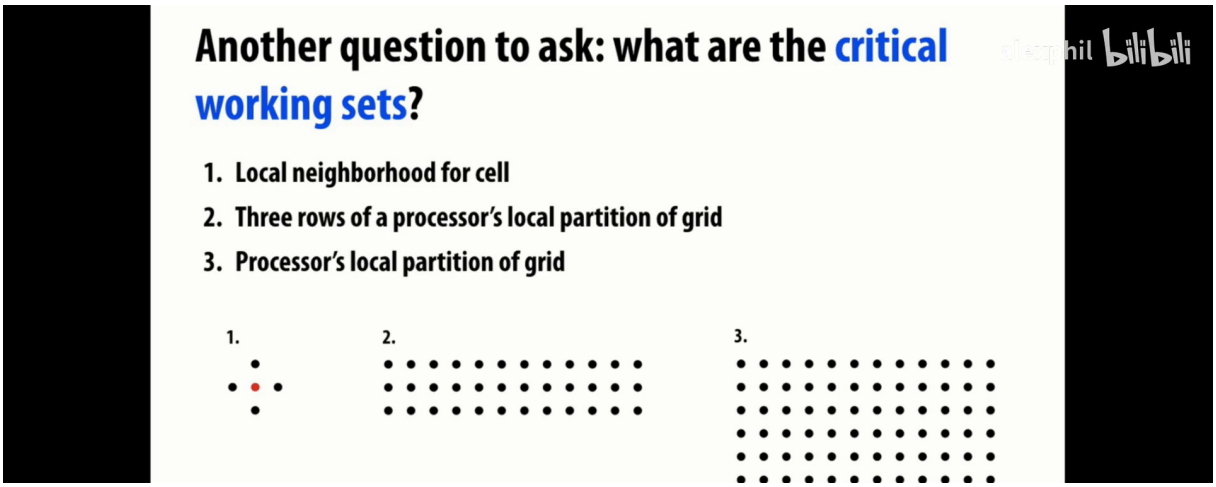


图 10: 关键问题：工作集有多大，局部邻域与整块瓦片[10]

讲者提醒”工作集”这个词用得很含糊——它指你在某段时间内需要的内存或数据规模。但工作集大小取决于所讨论的时间尺度。

在这个例子里，最短时间尺度下每个处理器需要访问一个网格点及其四个邻居。若我们在行内扫描，那就是一种工作集。更一般地，某个给定瓦片内所有节点的状态集合也可视为工作集。

而缓存层次正是为应对”工作集大小随时间变化”这一事实而设计的：短期内用很小很快的缓存，然后逐级更大更慢。

[10] 视频画面时间区间：00:11:30–00:11:45。若为裁剪图，时间区间仍对应原视频画面。

4.4 静态分配的适用条件与实测

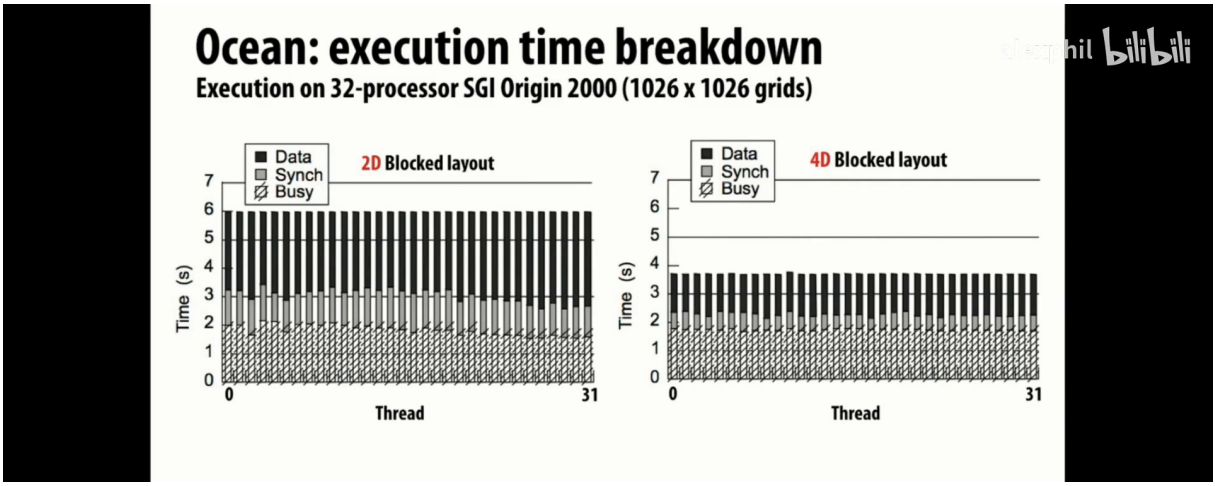


图 11: 海洋模拟的执行时间分解：32 处理器 SGI Origin 2000，1026×1026 网格[11]

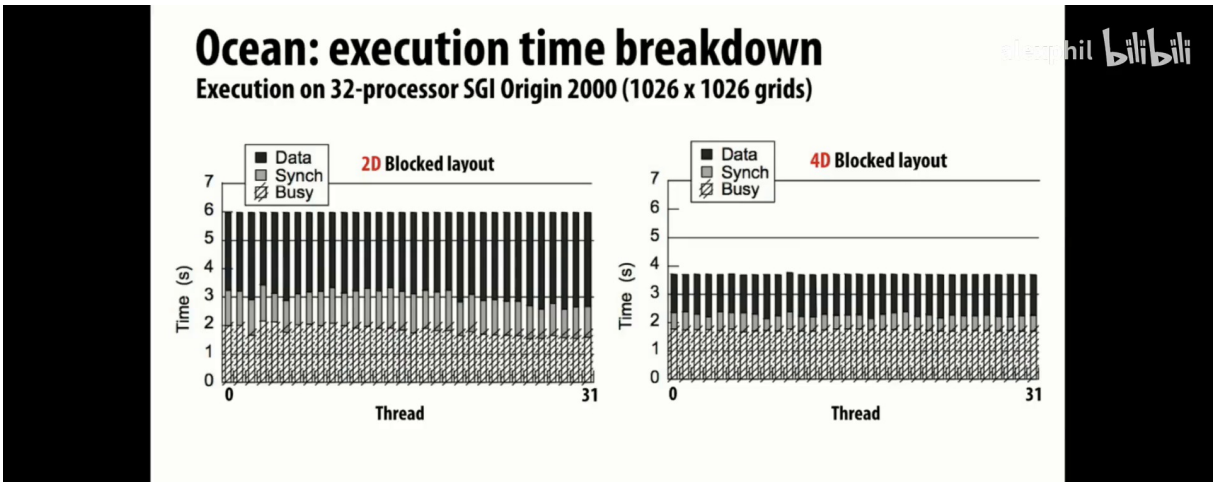


图 12: 执行时间分解续：各阶段占比[12]

讲者说明这是纯粹的静态分配，而它在这个应用上工作得很好，因为所有网格点基本上都在做同样的事，因此工作量在各瓦片间均匀分布。

分块形状的收益与机器类型无关

无论共享内存还是消息传递，计算正比面积、通信正比周长这条几何事实都成立。因此”用紧凑块划分”的结论可以跨机器类型迁移。差别只在于通信的实现方式与固定开销，而不在于最优形状。

他也确认这种被称作四维分块的做法确实改善了通信性能，而且它在分布式消息传递环境中同样表现极好，因为能获得相当好的算术强度。

[11] 视频画面时间区间：00:14:00–00:14:15。若为裁剪图，时间区间仍对应原视频画面。

[12] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

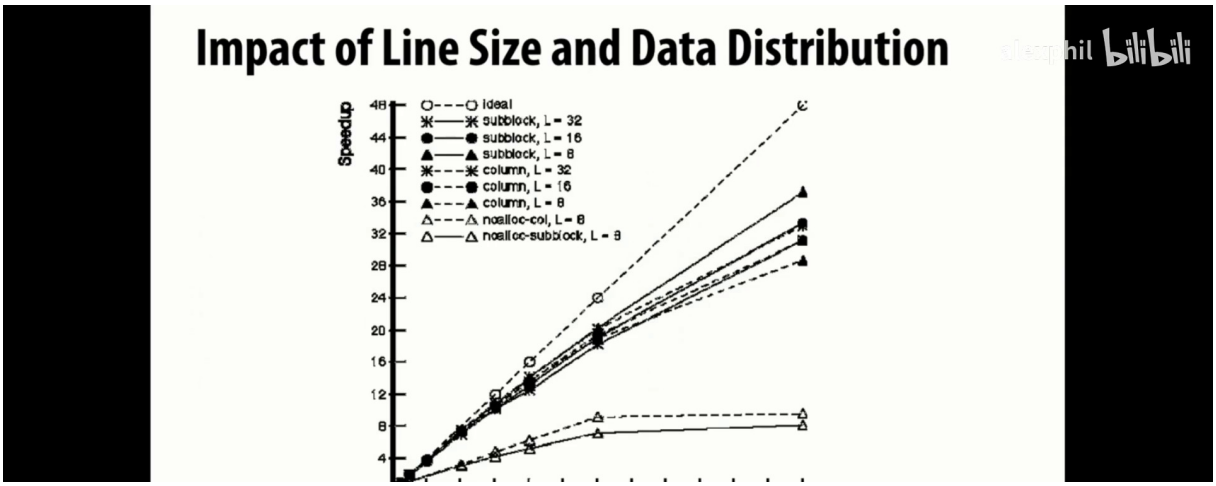


图 13: 缓存行大小与数据分布对加速比的影响[13]

他展示了一张关于加速比的图，说明不同内存分配机制下的差异：最上方是理想加速比（达不到），右下方则是完全没有局部性的糟糕分配，其余是各种分配选择形成的分布带。

讲者说自己认为这张图比他今天需要讲的更细，但它代表了一类典型问题。

他也指出一个重要连带效应：改善内存布局不仅减少通信时间，还能提供更规整的计算模式，从而避免因个别访问慢而拖累整体。

内存布局同时影响通信与抖动

好的布局既降低平均通信量，也降低通信时间的方差。在屏障同步的结构下，方差与均值同样重要——因为决定每步耗时的是最大值而非平均值。因此布局优化在同步型应用中的收益常大于其平均带宽改善所暗示的程度。

4.5 本章小结

网格划分的原则是让瓦片越方越好，因为计算正比面积、通信正比周长，方形使二者之比最大。屏障同步有两个代价：达成一致的时间，以及掉队者拖慢所有人。工作集大小取决于时间尺度，缓存层次正是为此设计。海洋模拟的静态分配之所以有效，是因为所有网格点工作量相同；四维分块确实改善了通信，在消息传递环境下同样有效。改善内存布局还能减少通信抖动。

[13] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

5 案例二：星系演化与 N 体问题

5.1 为什么比网格问题难

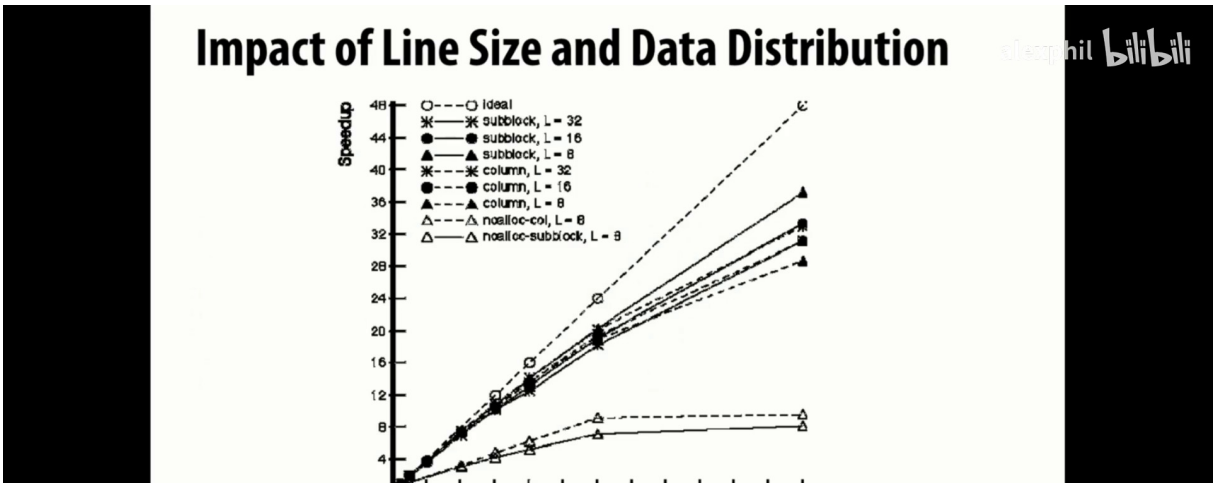


图 14: 星系演化：用 Barnes-Hut 算法建模引力相互作用[14]

讲者说更有挑战性的是星系模拟。基本设定是：把宇宙中所有恒星看作引力体，它们相互吸引，因此会形成轨道、旋臂等结构。我们要建模的是星系随时间的演化，而演化由这些恒星之间的全部相互作用决定。

困难在于：恒星在星系中并非均匀分布，它们以各种方式成团聚集。因此不能简单地把空间切成均匀网格、假设每块都一样。

5.2 $O(n^2)$ 与它的出路

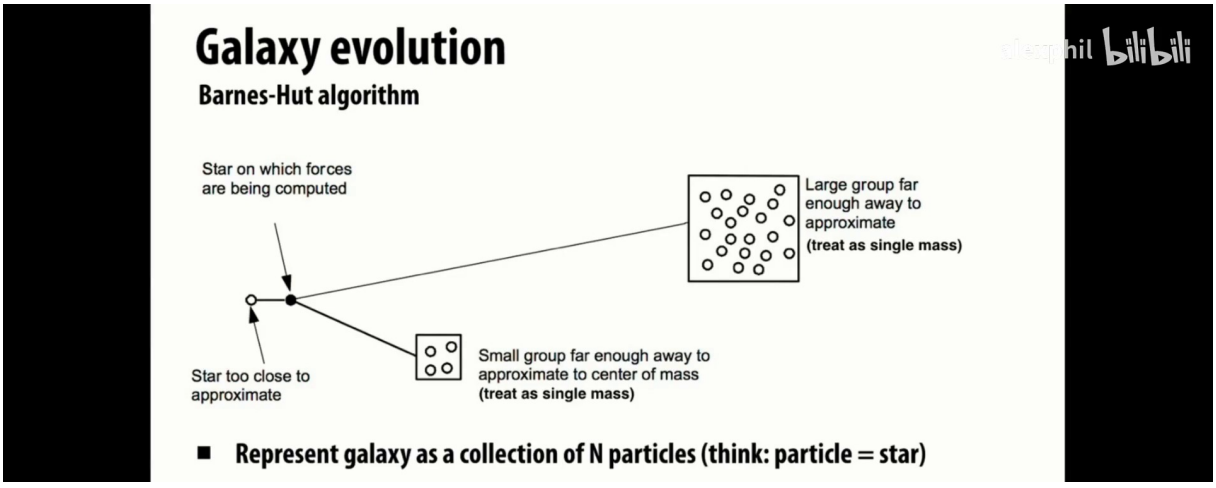


图 15: Barnes-Hut 算法：用远处星团的质心近似其引力影响[15]

朴素做法的问题是：对某一颗恒星，原则上必须计算其他 $n-1$ 颗星对它的作用力，因此是 n 平方的问题。虽然力是对称的，但那只能节省 2 倍。

[14] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

[15] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

关键观察是：如果某个星团足够远，就可以把它抽象成一个点质量——计算该远处星团的质心、把质量加起来，就得到它对目标恒星影响的相当好的近似。

于是可以利用”远处物体可被聚合”这一性质，把多个天体合并成单点模型。最终得到的算法大致是 $n \log n$ 而不是 n^2。

讲者特意限定：这不是最坏情况下的算法保证，而是对现实分布而言的实用结果。

是否把某星团当作点质量，由展开判据决定：

$$\frac{s}{d} < \theta$$

- s：该星团所在方格的边长；
- d：星团中心到目标恒星的距离；
- θ：阈值，通常取 0.5 左右；
- 满足则用质心近似，否则继续深入其子节点。

阈值越大越快但越不准，因此它同时是性能参数与精度参数。

近似是复杂度的来源，也是正确性的边界

Barnes-Hut 用”远处星团视作点质量”换取 $O(n \log n)$，这个近似的误差取决于星团张角与距离之比。因此它不是精确算法。工程上必须接受这个误差，并据此选择展开判据；若应用要求严格精度，就必须退回 $O(n^2)$ 或使用更高阶展开。

5.3 本章小结

星系模拟比网格问题难，因为星体分布不均而成团聚集，不能划分均匀网格。朴素做法是 $O(n^2)$，对称性只能省 2 倍。Barnes-Hut 的关键观察是远处星团可抽象为质心点质量，于是复杂度降到实用的 $O(n \log n)$，代价是引入近似。

6 Barnes-Hut 树

6.1 树的构造

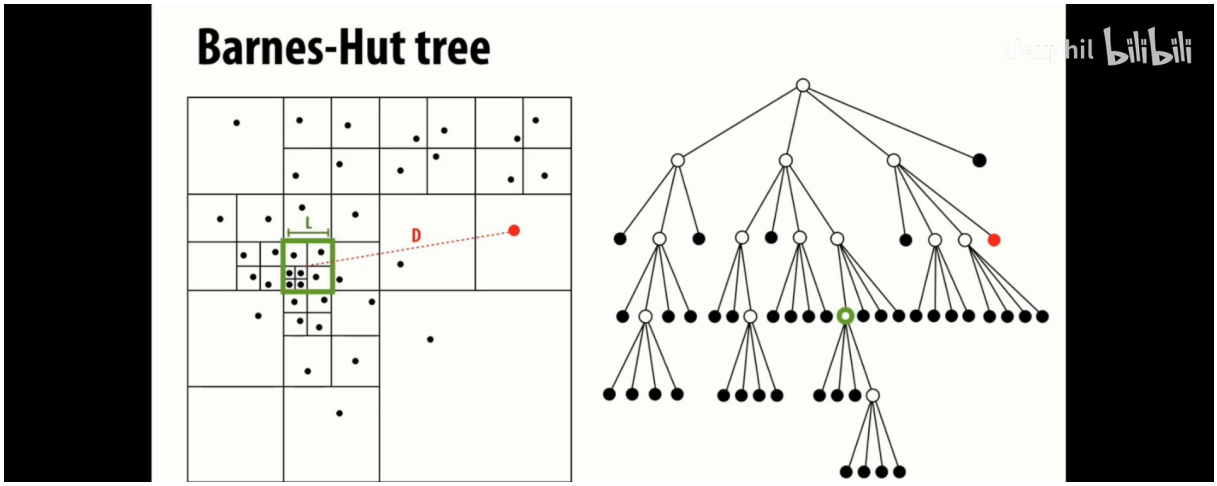


图 16: Barnes-Hut 树：反复二分空间直至每格只剩一颗星[16]

做法是把空间反复二分。讲者用二维演示（真实宇宙是三维，同样的思路在三维进行）。

图中不断把方格切得更细，直到某个方格中只剩一颗恒星——这就是最深的一层，它们会成为树的叶子。

这些方格构成一个层次结构，对应右侧那棵树，叫做四叉树：每个节点代表一块空间，它的四个子节点代表四个象限，如此递归。每一步在 x 与 y 方向各分一次。

他补充说，三维版本叫八叉树，是八路分裂，即在三个维度上各分一次。

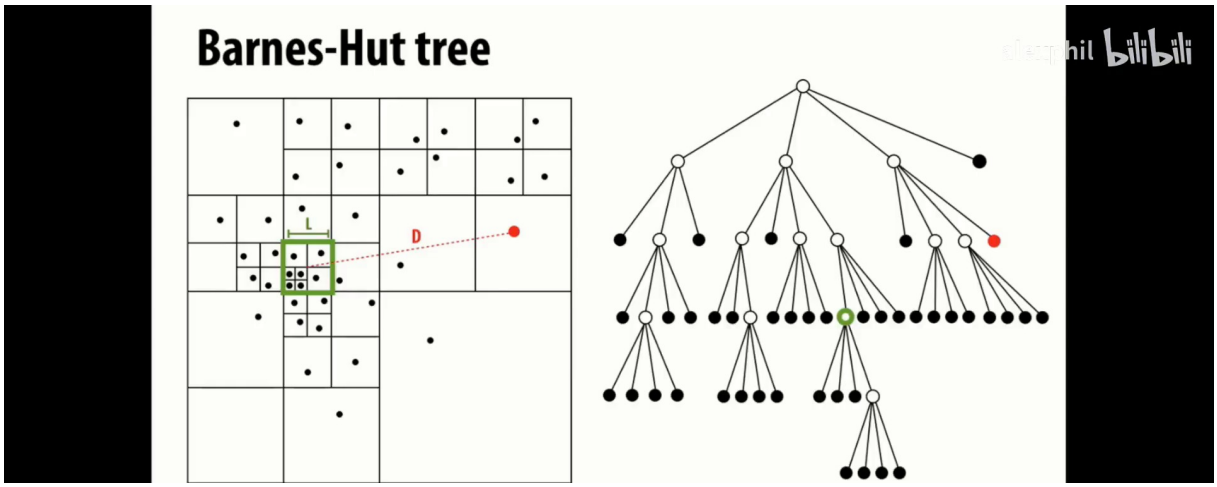


图 17: 树结构续：内部节点记录质心与总质量[17]

6.2 质心的逐层合成

这个结构提供了很好的封装方式：把一块区域内的信息聚合成紧凑形式，即单个点质量。

[16] 视频画面时间区间：00:21:10–00:21:25。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

黑色的节点是叶子，确实是单个点质量。而那些空白的内部节点，若计算该方格内所有质量的质心与总质量，那就是一个合理的代表。而且可以继续向上传播：基于四个（或八个）扇区的质心，计算逐级更粗方格的质心。

因此这个结构利用了引力本身的特性——质心性质。

质心聚合之所以有效，是因为引力按平方反比衰减

若相互作用随距离衰减得更慢，远处星团的内部结构就会显著影响结果，点质量近似便不成立。正因为引力衰减快，远处细节才可忽略。这个性质决定了树方法适用的相互作用类型，也是它不能直接套用到所有 n 体问题的原因。

6.3 建树与遍历的代价

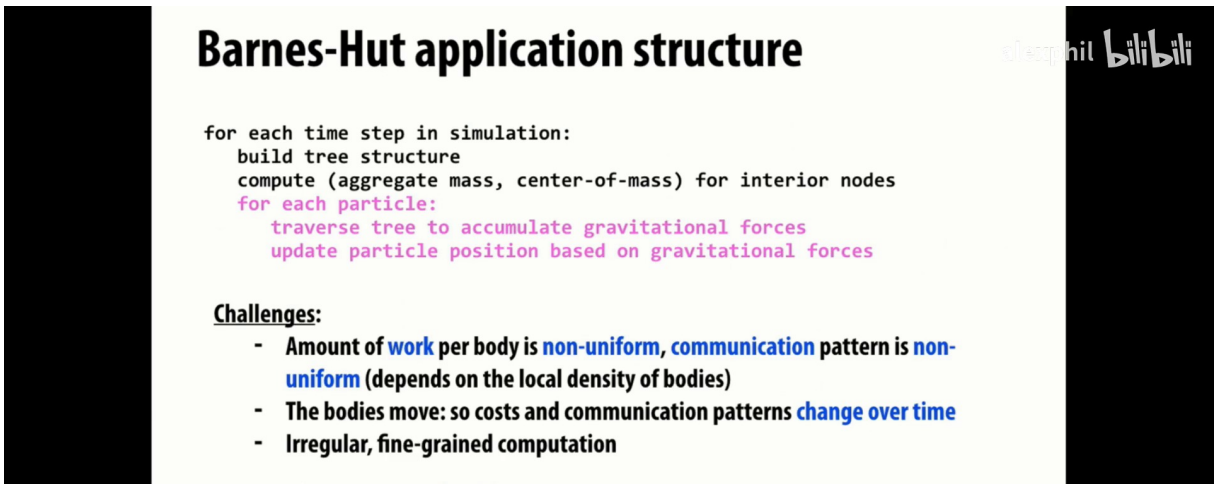


图 18: Barnes-Hut 的应用结构：每个时间步建树、计算力、更新位置[18]

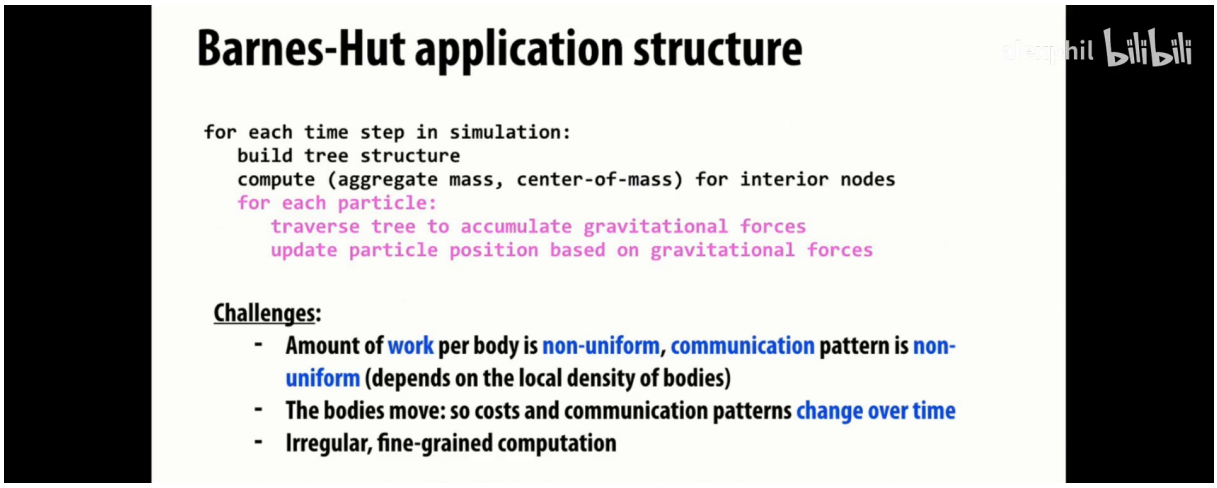


图 19: 应用结构续：建树、遍历、更新位置的循环[19]

讲者先讨论了建树。有学生问：如果只有少数恒星移动，是否真的需要重建整棵树？

[18] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

[19] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

讲者回答可以动态做：若只是把一颗星从 A 点移到 B 点，通常只是跨越相邻方格，可以更直接地处理。所以这当然可行。

但他说无论如何，假设我们建这棵树并完成聚合过程，可以看到它的代价只正比于树的节点数，也就是与天体数线性相关。因此这里有一个明显的并行来源。

然后是遍历。对树中每个叶子，都要计算作用在它上面的所有引力：有些纯粹是局部的，有些来自更远处。为此要从该节点向上走，再沿某些分支向下，有的分支走得比其他分支更深。

6.4 本章小结

Barnes-Hut 用反复二分把空间切成层次结构，二维为四叉树、三维为八叉树，叶子是单颗星，内部节点记录质心与总质量。质心可逐层向上合成，其有效性依赖引力的平方反比衰减。建树代价与天体数线性相关，因此有明显并行性；遍历则对每个叶子上下走树，深度因距离而异。

7 负载不均与动态划分

7.1 问题的形态

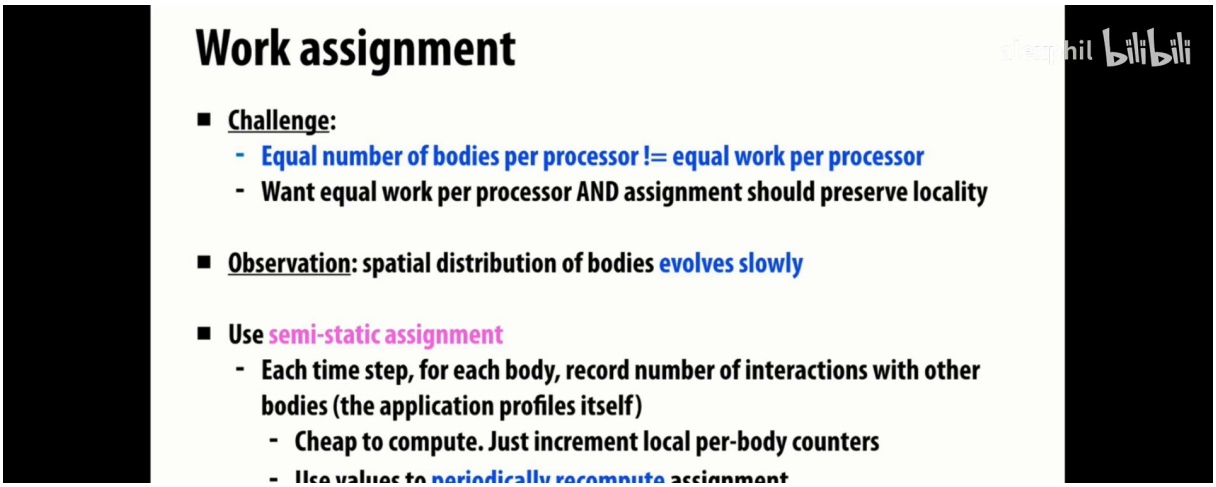


图 20: 工作分配：每个处理器分配相同数量的天体不等于相同的工作量[20]

讲者指出，对给定物体而言，尤其是考虑到分布如此不规整，工作量分布很不均匀，通信模式也不均匀。

但另一方面也不算太糟，因为我们主要是在这棵树上上下移动，而不是遍历整个宇宙。

他还提到一个随之而来的问题：若把恒星按空间分布到各处理器上，那么随着恒星移动，这个分配可能需要改变。好消息是这些问题都可控，而且由于树带来的局部性，大部分通信会相当局部。

7.2 不能按数量分配

于是引出一个有趣的问题：应该如何把工作分配给处理器？

讲者的回答很明确：由于工作量在天体间并不均匀，有些需要更多工作，所以实际做法是在运行中动态测量每个天体需要多少工作量，然后据此管理，使各处理器承担大致相同的工作量——但这并不意味着给它们相同数量的天体。

[20] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

负载均衡的目标是等时间，不是等数量

在规整网格上，等数量恰好等于等时间，因此静态分配就够。在树结构上，不同叶子访问的节点数相差很大，等数量会导致等时间严重失衡。因此判据必须换成”每个处理器的预计耗时相等”，而这需要运行时测量。

7.3 用代价区域分配

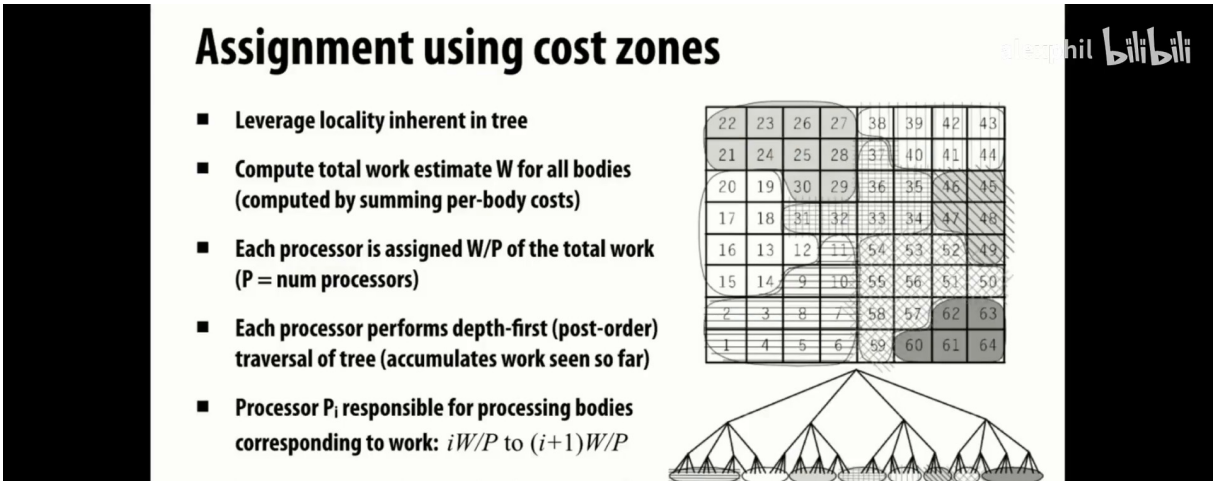


图 21: 用代价区域做分配：利用树固有的局部性[21]

讲者展示了具体做法：由于四叉树划分，可以把树的一部分分配给处理器，这在空间上对应把星系或宇宙切成一些区域——这些区域不是方形或矩形，但每一个都连续。

这是个有意思的处理不均匀性的方式：既保留了一定的空间局部性，又能通过相当简单的算法得到相当均匀的划分。

近似划分的代价必须与均衡收益一起评估

划分本身要花时间，且划分越频繁收益越小。若每步都重新划分，划分开销可能超过均衡带来的收益。因此实际系统采用”半静态”策略：在一段时间内固定分配，只在失衡累积到一定程度时才重划。

他坦率地说，找到完美划分很可能是 NP 完全问题，但可以设想各种分治算法，它们会相当有效。

这种被称为半静态的方案是：在一段时间内让处理器 1 处理横条纹那一组，处理器 2 处理下一组，依此类推。

7.4 工作集视角下的树遍历

讲者再次回到工作集的概念。以图中那个红色节点为例，它需要遍历以找出所有作用在它上面的力，因此会沿树上下走到不同深度，访问树的一些部分，而这些部分在空间上可能并不局部。

但访问的总量相当有限，因此若有任何形式的缓存，表现会比较好——因为沿树上下时访问的树的部分会被复用。

[21] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

树的局部性是”结构局部性”而非”空间局部性”

遍历 Barnes-Hut 树时，一个叶子访问的节点在几何空间上可能分散，但在树结构上集中在少数路径及其子树。因此缓存仍能受益，因为上层节点被大量叶子共享。理解这一点才能解释为什么这种看似”乱跳”的访问模式实测表现并不差。

7.5 本章小结

树结构导致工作量与通信模式都不均匀，因此不能按天体数量平均分配，而要在运行中动态测量每个天体的工作量。用代价区域分配可以利用四叉树的空间连续性，得到连续但非矩形的区域，既保留局部性又相当均衡；完美划分很可能是 NP 完全问题，但分治算法足够有效。树的局部性是结构性的，因此缓存表现仍好。

8 总结与延伸

讲者在结尾点出一个一般性结论：这些方法并不完全依赖科学领域或问题领域，而更多与问题的一般特征有关，而不是与你要解决的具体问题有关。

把全讲串起来，这是一条从原理到案例的落地路径。这一讲的两个核心关切是负载均衡与通信开销。硬件假设是 NUMA 共享内存：部分内存访问更快，跨芯片要经对方 DRAM 控制器。海洋模拟把空间离散成二维网格，按小时步更新状态；提高分辨率必须同时缩短时间步，因此分辨率翻倍要 16 倍算力。网格划分的原则是让瓦片越方越好，因为计算正比面积、通信正比周长；屏障同步要付出达成一致的时间，且掉队者会拖慢所有人；工作集大小取决于时间尺度，缓存层次正为此设计。海洋模拟的静态分配有效，因为所有网格点工作量相同；四维分块确实改善通信，在消息传递环境下同样有效。星系模拟因星体成团而更难：朴素做法 $O(n^2)$，对称性只省 2 倍；Barnes-Hut 用远处星团可聚合为质心这一观察，把复杂度降到实用的 $O(n \log n)$，代价是近似。树在二维为四叉树、三维为八叉树，叶子是单星，内部节点记录质心与总质量，质心逐层向上合成。建树代价与天体数线性相关，遍历深度因距离而异，因此工作量不均，必须动态测量并按代价区域划分，而不是按数量平均。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 两个案例的差别不在应用领域，而在”工作量是否可预知”。网格问题的工作量均匀且静态可知，因此静态划分足够；树结构问题的工作量取决于运行时分布，因此必须动态测量。这是划分策略选择的真正判据。
- 分辨率与算力的四次方关系是所有物理模拟的共同约束，不是某个应用的特殊困难。理解这一点，才能理解为什么这类问题需要超算，以及为什么它们对硬件演进极其敏感。
- 屏障同步把最慢者的延迟放大为全局延迟。因此在同步型应用中，通信时间的方差与均值同等重要，甚至更重要。这解释了为什么布局优化在这类程序中的收益常超出带宽改善所暗示的程度。
- Barnes-Hut 的 $O(n \log n)$ 是实用复杂度而非最坏情况保证，且以近似为代价。因此它不是”更好的算法”，而是”在可接受误差下更快的算法”。评估这类方法时必须同时报告精度与耗时。

这堂课的边界在于它是案例综述：讲者明确说部分图表比他今天需要讲的更细，也未深入具体实现的调优细节。

两个可以直接从本讲数据提出的下一步问题：其一，代价区域划分假设工作量在短时间内变化缓慢，那么当星体分布剧烈变化时，重新划分的频率应如何确定？其二，Barnes-Hut 的近似误差由展开判据控制，那么在并行环境下，不同处理器采用不同判据是否会造成结果依赖线程调度？